Redes na prática

Aprendendo TCP, UDP e Python com seu port scanner

Uma apostila de estudo baseada nos arquivos reais deste projeto: connection.py, main.py e test_connection.py. O objetivo não é decorar chamadas de socket: é conseguir prever o que o programa fará, interpretar o que a rede respondeu e reconhecer o que não foi possível descobrir.

Versão estudada: a implementação completa corrigida foi preservada em docs/snapshot/. Durante a produção, o main.py da raiz foi reduzido ao menu; essa edição foi mantida. As explicações e o apêndice usam o snapshot completo, não afirmam que o main.py atual ainda executa todo o scan. connection.py e test_connection.py foram copiados sem alterações para acompanhar essa versão.

Escopo: IPv4, TCP connect scan e sondagem UDP genérica, sequenciais. O projeto não implementa SYN scan, identificação de versão, descoberta de vulnerabilidades, IPv6 nem captura própria de ICMP.

Regra de laboratório: pratique em 127.0.0.1 ou em ambientes para os quais tenha autorização explícita. Uma sondagem pode aparecer em logs e uma mensagem UDP pode acionar comportamento de uma aplicação. Porta aberta não é autorização para acessar um serviço.

Como estudar: leia os fundamentos, acompanhe o código numerado no apêndice, execute o laboratório e responda às perguntas antes de abrir o gabarito. Os exemplos de tráfego são esquemas conceituais, não capturas reais.

Roteiro de estudo

O sumário e os marcadores do PDF permitem voltar diretamente aos capítulos. Leia os fundamentos antes de interpretar os resultados do laboratório.

1. O que você está realmente medindo 3
2. Camadas: do Python até o fio 4
3. IP, prefixo, rota e loopback 5
4. Portas e sockets: os pontos de comunicação 6
5. TCP: o que acontece ao conectar 7
6. UDP e ICMP: aprender a interpretar silêncio 8
7. DNS e resolução de nomes 9
8. Estados, firewall e timeout 10
9. Visão geral dos arquivos 11
10. main.py explicado por blocos 12
11. connection.py: menu e TCP 13
12. connection.py: UDP e exceções 14
13. Por que os testes são importantes 15
14. Laboratório TCP: enxergue o serviço abrir e fechar 16
15. Laboratório UDP: uma porta aberta pode ficar calada 17
16. Observação de tráfego e roteiro de diagnóstico 18
17. Limitações reais e caminho de evolução 19
18. Exercícios de entendimento 20
19. Gabarito comentado 21
20. Referências e revisão rápida 22
Apêndice A — main.py completo 23
Apêndice B — connection.py completo 24
Apêndice C — testes completos 25
Apêndice D — servidor TCP local 26
Apêndice E — servidor UDP local 27
Apêndice F — cliente do laboratório 28
Apêndice G — evidências de execução 29

1. O que você está realmente medindo

Um port scanner envia estímulos e observa efeitos. Ele não abre uma janela para dentro da máquina remota. A conclusão depende do protocolo, da resposta recebida, do prazo de espera e do ponto da rede onde você está.

Um serviço geralmente precisa de um processo, um socket associado a um endereço e uma porta, e uma política de rede que permita a comunicação. Se alguma dessas condições mudar, o mesmo destino pode apresentar outro resultado.

Porta aberta: houve evidência compatível com uma aplicação acessível naquele protocolo. No TCP do projeto, essa evidência é o estabelecimento de conexão. No UDP, é o recebimento de um datagrama pelo socket associado ao destino.

Porta fechada: houve uma recusa compatível com ausência de serviço ou uma rejeição ativa. No TCP, o programa interpreta ECONNREFUSED como closed. Em redes reais, um firewall também pode gerar uma rejeição: o rótulo é uma inferência operacional, não uma inspeção da lista de processos.

Resultado inconclusivo: não chegou evidência suficiente. Timeout não separa serviço silencioso, filtragem, perda, congestionamento ou indisponibilidade.

Uma regra importante para segurança: acessibilidade, identidade do serviço e vulnerabilidade são três perguntas diferentes. Conectar à porta 443 responde apenas à primeira. Não demonstra HTTPS, uma versão específica, uma falha ou a legitimidade do servidor.

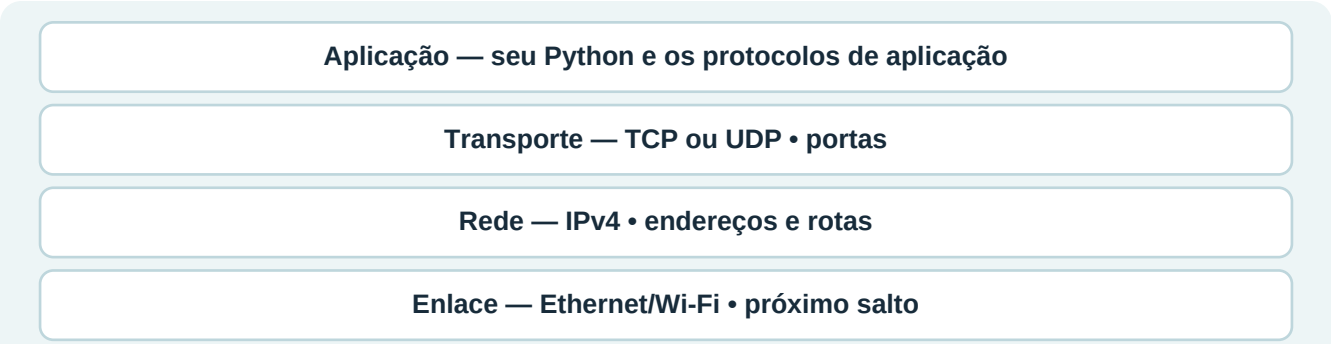
Pense em três colunas

Pergunta	Exemplo	O projeto responde?
Consigo estabelecer TCP?	Conectar a 127.0.0.1:8000	Sim
Qual protocolo de aplicação?	HTTP ou outro serviço?	Não identifica
Existe vulnerabilidade?	Falha de autenticação?	Não testa

Pergunta de controle: por que duas pessoas, em redes diferentes, podem obter estados diferentes para a mesma porta? Guarde sua hipótese: roteamento, firewall e endereço de escuta aparecem nos próximos capítulos.

2. Camadas: do Python até o fio

A divisão em camadas ajuda a localizar responsabilidades. Seu código trabalha com a interface de sockets; o sistema operacional implementa a maior parte do transporte, da rede e da transmissão pela interface.



Esquema conceitual • não representa uma captura de tráfego

Aplicação: decide o significado dos dados. HTTP, DNS e seu servidor de eco são protocolos ou comportamentos de aplicação. A mensagem b'probe' pertence a essa camada; UDP não sabe o que ela quer dizer.

Transporte: TCP oferece um fluxo de bytes ordenado e mecanismos de confiabilidade; UDP oferece datagramas sem garantia de entrega, ordem ou eliminação de duplicatas pelo próprio protocolo.

Rede: IP identifica destinos e permite encaminhamento entre redes. Entrega é de melhor esforço. Roteadores decidem o próximo salto usando rotas; eles não precisam entender o Python do cliente.

Enlace: Ethernet e Wi-Fi entregam quadros no trecho local. Em IPv4 sobre Ethernet, ARP pode resolver o endereço MAC do próximo salto. Esse próximo salto pode ser o próprio destino local ou o gateway.

Encapsulamento

Ao transmitir, dados de aplicação recebem um cabeçalho TCP ou UDP, depois um cabeçalho IP e finalmente o enquadramento do enlace. No recebimento, o processo é desfeito e os dados são encaminhados ao socket apropriado.

Os termos têm contextos diferentes: segmento TCP, datagrama UDP, pacote IP e quadro Ethernet. No dia a dia, 'pacote' costuma ser usado de modo genérico, mas distinguir as camadas ajuda a ler uma captura.

Limite do modelo: no loopback, 127.0.0.1, não há travessia física de Wi-Fi/Ethernet nem necessidade de ARP para chegar a outra máquina. O sistema operacional entrega o tráfego localmente. O laboratório usa justamente esse caminho para reduzir variáveis.

3. IP, prefixo, rota e loopback

Um IPv4 tem 32 bits e costuma ser escrito em quatro grupos decimais, como 192.168.1.10. Um prefixo CIDR descreve quantos bits pertencem à parte de rede: em 192.168.1.0/24, os primeiros 24 bits formam o prefixo.

O sistema consulta a tabela de rotas para decidir como alcançar um destino. Uma rota específica pode levar a uma interface local; a rota padrão atende destinos sem uma rota mais específica. Não basta uma string parecer um IP válido: precisa haver caminho e permissão.

IP privado e IP público: faixas como 10.0.0.0/8, 172.16.0.0/12 e 192.168.0.0/16 são usadas em redes privadas e não são globalmente roteáveis na Internet pública. Máquinas de redes diferentes podem ter o mesmo endereço privado sem representar o mesmo destino.

NAT: um equipamento pode traduzir endereços e portas. Um serviço escutando no seu PC não se torna automaticamente acessível pelo IP público do roteador. Encaminhamento de porta, firewall e a rede do provedor também interferem. Não é necessário configurar isso para estudar este projeto.

127.0.0.1: endereço de loopback IPv4. O cliente e o servidor estão no mesmo host. 'localhost' é um nome normalmente resolvido localmente, mas não é o mesmo conceito de um endereço literal.

0.0.0.0 no bind: representa todas as interfaces IPv4 locais no contexto de escuta. Não é o endereço que você deve usar como destino do laboratório. Prefira bind em 127.0.0.1 para não expor os exemplos à rede.

Diagnóstico somente de leitura no Linux

```
ip addr
ip route
ss -lnt
ss -lnu
```

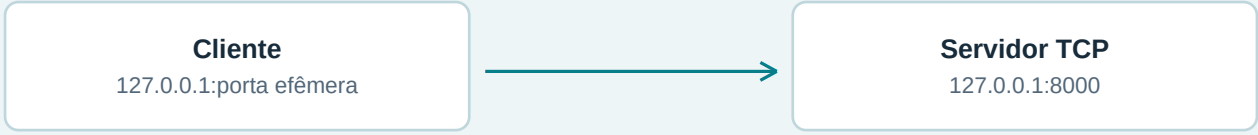
Os dois primeiros mostram endereços e rotas. O terceiro lista sockets TCP em escuta; o quarto, sockets UDP associados. Essas ferramentas são externas ao Python e podem não estar instaladas em todo ambiente. Use-as para comparar o que o sistema tem com o que o scanner observa.

4. Portas e sockets: os pontos de comunicação

Uma porta de transporte é um campo de 16 bits. Para sondagens de serviços comuns, valide entradas entre 1 e 65535. A porta 0 tem significado especial em APIs de bind: permite pedir ao sistema uma porta disponível. Por isso os testes usam bind(('127.0.0.1', 0)).

TCP e UDP têm espaços de portas distintos. Um serviço TCP na porta 53 não implica um serviço UDP na porta 53; o número coincide, o protocolo não. DNS é um exemplo conhecido de aplicação que utiliza ambos, conforme a operação.

Um endpoint combina endereço e porta. Para raciocinar sobre uma conversa de transporte, considere protocolo, IP de origem, porta de origem, IP de destino e porta de destino. Não pense apenas no número da porta remota.



Esquema conceitual • não representa uma captura de tráfego

O cliente normalmente não escolhe sua porta de origem: o sistema aloca uma porta efêmera. O servidor, por outro lado, faz bind em uma porta conhecida ou configurada. A faixa efêmera depende do sistema e da configuração.

Socket não é a própria rede

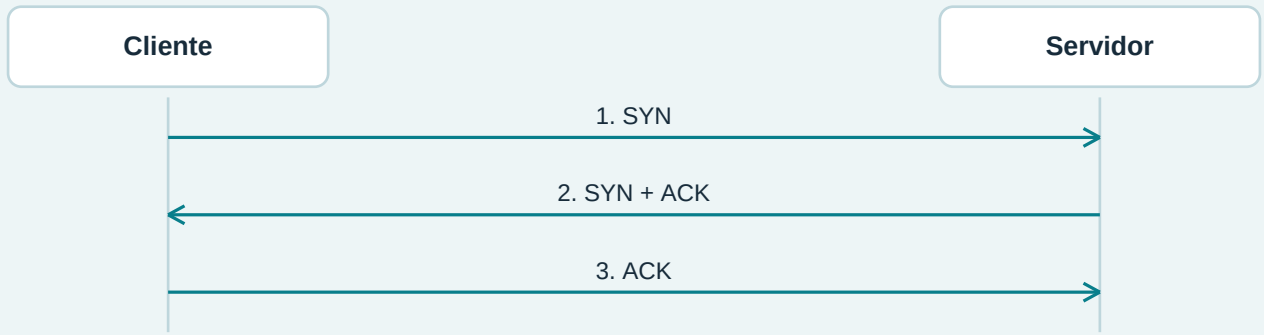
socket.socket(...) cria um recurso local controlado pelo sistema operacional. Isso não significa que o servidor existe, que um pacote foi entregue ou que uma conexão foi estabelecida.

Chamada	Responsabilidade principal
socket()	Criar o endpoint local
bind()	Associar endereço/porta local
listen()	Preparar aceitação TCP
accept()	Obter socket de uma conexão TCP
connect()	Iniciar TCP ou associar destino UDP
send()/recv()	Enviar/receber dados no socket
close()	Liberar o recurso local

A mesma chamada pode ter semântica diferente conforme o tipo do socket. A diferença de connect() em TCP e UDP é central para não tirar conclusões falsas.

5. TCP: o que acontece ao conectar

TCP é orientado a conexão. O cliente e o servidor estabelecem estado antes da troca normal de dados. O esquema simplificado do estabelecimento é o three-way handshake.



Esquema conceitual • não representa uma captura de tráfego

SYN: o cliente solicita o estabelecimento. **SYN+ACK:** o servidor reconhece e apresenta sua própria sequência. **ACK:** o cliente reconhece a resposta. Na operação normal, connect_ex() bem-sucedido indica que a conexão foi estabelecida pela pilha TCP.

Não é seu código Python que monta manualmente esses cabeçalhos. O kernel gerencia flags, números de sequência, retransmissões, buffers e estados. Seu programa solicita a operação e lê o resultado.

Três situações para o scanner

Se há um servidor TCP acessível em escuta, a conexão pode completar e connect_ex() retorna 0. Se o destino recusa a conexão, é comum haver um RST e aparecer ECONNREFUSED. Se os pacotes são descartados, a tentativa pode terminar em timeout.

O scanner implementa um **TCP connect scan**: completa uma conexão normal e depois fecha o socket. Não é um SYN scan que envia pacotes crus e decide o resultado antes de completar o handshake.

TCP é fluxo, não mensagens

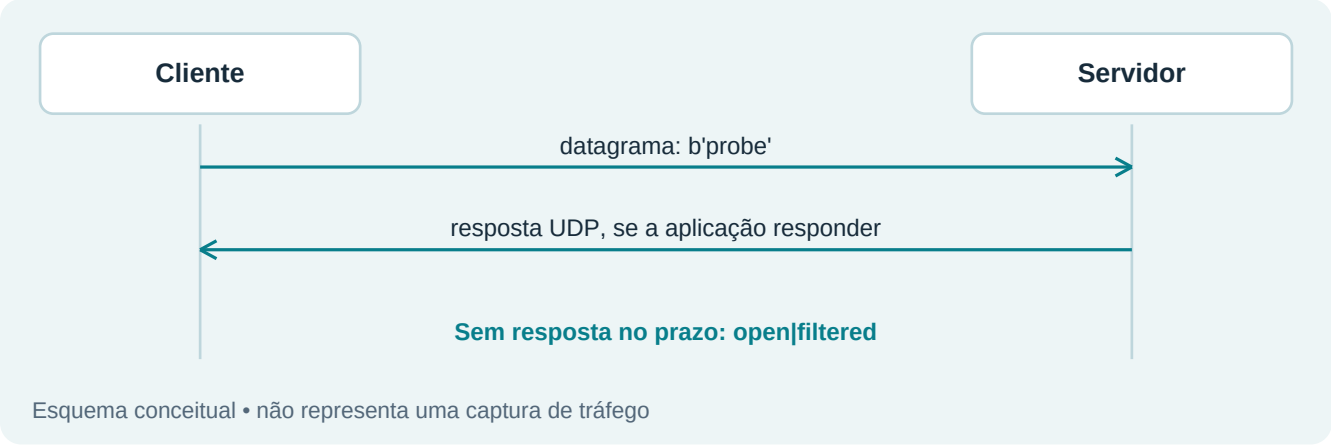
Depois da conexão, TCP preserva a ordem dos bytes, não as fronteiras de cada send(). Dois envios podem ser recebidos juntos ou repartidos. Uma aplicação precisa definir enquadramento: tamanho prefixado, delimitador ou outra regra. Nosso scan TCP não envia conteúdo de aplicação, por isso não precisa desse enquadramento.

Conexão estabelecida não quer dizer que o processo já chamou accept(), nem que uma requisição seria aceita. A fila de conexões do kernel pode concluir o handshake antes do processamento pela aplicação. O teste de TCP aberto aproveita isso.

Confiabilidade não é invencibilidade: TCP tenta recuperar perdas enquanto a conexão é viável. Ainda pode falhar por timeout, reset, falta de recursos ou ruptura de rede. Também não oferece criptografia por si só; HTTPS acrescenta TLS sobre TCP em seu uso tradicional.

6. UDP e ICMP: aprender a interpretar silêncio

UDP preserva fronteiras de datagramas. Um envio gera uma mensagem de transporte, mas o protocolo não garante entrega, ordem ou retransmissão. Se a aplicação precisa dessas propriedades, deve providenciá-las ou usar outro protocolo apropriado.



Esquema conceitual • não representa uma captura de tráfego

connect() em UDP não faz handshake

No socket UDP conectado do projeto, connect((target_ip, port)) associa o destino para que send() e recv() possam ser usados sem repetir o endereço. Também restringe a recepção ao par remoto associado conforme a API. Não há verificação remota equivalente ao handshake TCP.

send(b'probe'): pede o envio de um datagrama com bytes da aplicação. O b indica bytes, não str. O sucesso de send() não comprova que a mensagem chegou ao destino.

recv(4096): espera um datagrama e permite receber até esse limite de bytes. Um datagrama maior pode ser truncado. O programa só usa a presença de resposta, não seu conteúdo. Mesmo um datagrama vazio é uma resposta; o código não depende de sua conversão para verdadeiro/falso.

ICMP não é uma resposta UDP

Se não há serviço na porta, o destino pode enviar ICMP Destination Unreachable / Port Unreachable. Em IPv4, esse caso é type 3, code 3. O sistema pode traduzir a informação em ConnectionRefusedError no socket UDP. A entrega desses erros à aplicação depende do sistema operacional e da situação.

O programa não captura ICMP diretamente, não analisa todos os tipos/códigos e não correlaciona mensagens por conta própria. Outros erros seguem para o tratamento de OSError do main.py.

Por que o silêncio é ambíguo? Pode haver firewall, perda, ICMP limitado por taxa, host indisponível ou uma aplicação que ignorou b'probe'. DNS, por exemplo, espera uma mensagem estruturada; bytes arbitrários não constituem uma consulta válida.

Por isso open|filtered quer dizer 'não consegui distinguir aberto de filtrado com esta sondagem'. Não é uma garantia de que apenas essas duas causas sejam fisicamente possíveis.

7. DNS e resolução de nomes

Em main.py, o programa executa:

```
target = input('Insert the target IP or hostname: ').strip()
target_ip = socket.gethostbyname(target)
```

O alvo digitado pode ser um IPv4 ou um nome. gethostbyname() pede resolução IPv4 ao mecanismo do sistema. Esse mecanismo pode consultar arquivo hosts, cache, DNS ou outras fontes configuradas. Resolver um nome não significa que toda consulta foi enviada pela rede.

DNS traduz nomes em informações de endereçamento; não testa portas. Resolver um nome com sucesso não demonstra que o host responde TCP ou UDP. Falhar na resolução também não é evidência de porta fechada: nenhuma sondagem daquele destino chegou a acontecer.

Nomes podem corresponder a vários endereços. Esta função retorna um IPv4, e o scanner usa somente esse resultado. Ele não percorre todos os endereços, não resolve IPv6 e não controla qual endereço um balanceador fornecerá.

Relação com os protocolos

DNS tradicional usa UDP e também TCP na porta 53. Não reduza a regra a 'DNS é UDP': tamanho da resposta, truncamento e tipo de operação podem exigir TCP. DNS criptografado também pode usar outros transportes e portas.

No código atual, 'UDP/53 open|filtered' não comprova que não há DNS. O payload b'probe' não segue o formato de uma consulta DNS. Uma futura sondagem específica teria de gerar uma mensagem válida e conferir a correspondência da resposta.

Erro tratado no projeto

socket.gaierror é capturado no bloco externo de main(). O programa imprime a falha e retorna 1. O trecho de entrada usa strip() para retirar espaços nas extremidades, mas não implementa validação semântica completa do alvo.

O TIMEOUT do projeto é aplicado aos sockets das sondagens. Ele não define um prazo global para a resolução de nome nem para todo o programa. Essa distinção importa ao medir duração.

8. Estados, firewall e timeout

Evidência observada	Estado no projeto	Limite da conclusão
TCP connect_ex() == 0	open	Não identifica aplicação
TCP ECONNREFUSED	closed	Rejeição pode vir de política
TCP timeout/EAGAIN	filtered/unknown	Não prova filtragem
Resposta no socket UDP	open	Conteúdo não é validado
UDP ConnectionRefusedError	closed	Depende do erro entregue pelo SO
UDP sem resposta no prazo	open|filtered	Pode ser serviço silencioso
Outro erro	error: ...	Não deve virar closed

Um firewall pode **descartar** tráfego sem resposta ou **rejeitar** ativamente. Também pode aplicar regras por endereço de origem, protocolo, estado da conexão, interface ou direção. Isso explica resultados diferentes de dentro e de fora de uma rede.

O timeout é uma decisão do observador

settimeout(1.0) limita operações bloqueantes naquele socket. Não muda a disponibilidade do serviço. Reduzir o prazo torna o scan mais rápido, mas pode aumentar resultados inconclusivos. Aumentá-lo dá mais chance a respostas lentas, sem garantir uma conclusão.

A versão antiga chamava setdefaulttimeout() depois de criar o socket. Essa configuração global só fornece o padrão para sockets criados posteriormente: não atualiza o socket anterior. O código atual configura cada socket diretamente antes da operação.

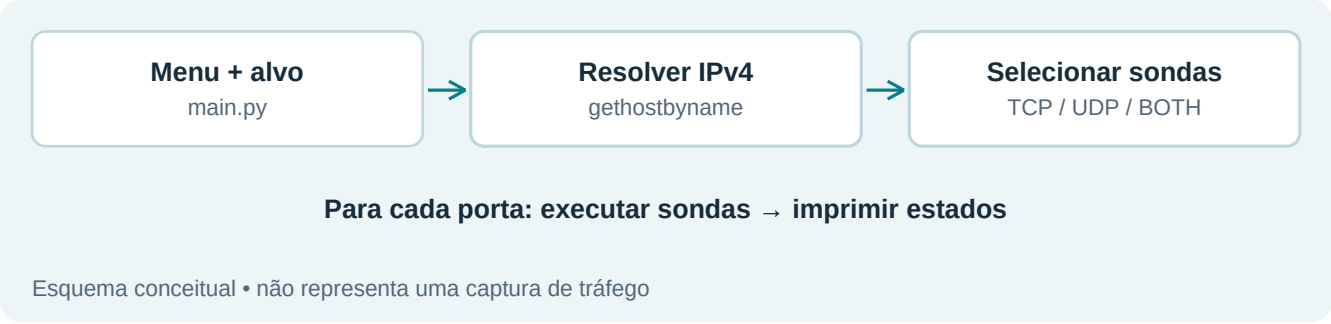
Duração do scan sequencial

A lista tem sete portas. TCP ou UDP fazem sete sondagens; BOTH faz quatorze, pois cada porta é testada em dois protocolos. Um cenário ilustrativo em que cada sondagem consome aproximadamente um segundo daria cerca de 14 segundos em BOTH, mais custos adicionais.

Isso não é um limite rígido: DNS e outros custos ficam fora; UDP pode aguardar em operações distintas; recusas imediatas terminam antes; o agendamento também varia. Meça tempo real com time.perf_counter() em vez de prometer duração exata.

Pergunta: reduzir TIMEOUT para 0.1 transforma open|filtered em closed? Não: muda a janela de observação, não elimina a ambiguidade.

9. Visão geral dos arquivos



Esquema conceitual • não representa uma captura de tráfego

main.py organiza a execução. Obtém escolha e alvo, resolve o endereço, seleciona funções, percorre portas e imprime resultados. As constantes PORTS e TIMEOUT são a configuração atual.

connection.py contém a interface de seleção e as sondagens. connection_choice() retorna um protocolo normalizado; connection_tcp() e connection_udp() recebem endereço, porta e timeout, e retornam um estado textual.

test_connection.py verifica o comportamento. Combina sockets reais no loopback com substituições controladas para testar o menu e o despacho de protocolos sem depender de serviços externos.

Contratos úteis

```
connection_choice() -> 'TCP', 'UDP' ou 'BOTH'
connection_tcp(target_ip, port, timeout) -> estado
connection_udp(target_ip, port, timeout) -> estado
```

As anotações acima descrevem contratos de estudo; não são declarações presentes no código. A implementação usa strings, sem Enum nem dataclass.

As sondagens não imprimem o resultado nem perguntam o alvo. Essa separação permite usá-las em testes e, futuramente, em outra interface. O main.py decide como apresentar o retorno.

Imports pertencem ao módulo

Importar socket em main.py não disponibiliza o nome socket dentro de connection.py. Cada arquivo precisa importar o que usa. connection.py importa errno e socket; main.py importa socket e connection.

Não há dependências externas para executar o scanner e seus testes: eles usam a biblioteca padrão. As ferramentas usadas para produzir esta apostila são separadas do programa.

10. main.py explicado por blocos

A numeração abaixo se refere ao snapshot completo no apêndice. Se você editar o arquivo, as linhas podem mudar.

Linhas 6–7 — configuração. PORTS mantém sua lista original. TIMEOUT é passado a cada sondagem. Maiúsculas são uma convenção para constantes; Python não impede reatribuição.

Linhas 10–19 — entrada e resolução. main() agrupa a execução. O try externo abrange a interação e o scan, permitindo tratar resolução de nome e interrupção em um lugar. O IP resolvido é mostrado para deixar explícito qual endereço será sondado.

Linhas 21–25 — lista de funções. probes guarda tuplas (nome_do_protocolo, função). O objeto connection.connection_tcp é armazenado sem parênteses: ainda não é executado. Em BOTH, entram as duas funções.

```
probes.append(('tcp', connection.connection_tcp))
```

Esse padrão evita copiar o mesmo laço para cada protocolo. Depois, probe(target_ip, port, TIMEOUT) executa a função selecionada. Funções podem ser valores em listas e tuplas.

Linhas 27–33 — laços e impressão. O laço externo escolhe a porta. O interno executa cada protocolo habilitado. Em BOTH, a ordem é TCP e UDP da primeira porta, depois TCP e UDP da segunda, e assim por diante. É tudo sequencial.

O except OSError interno transforma um erro operacional daquela tentativa em uma linha de saída e permite tentar as próximas. Isso não deve ser confundido com sucesso: error continua sendo error.

Linhas 35–41 — término. Falha de resolução retorna 1. Ctrl+C e fim da entrada retornam 130. Esse último é uma convenção escolhida pelo programa; EOF não é necessariamente um sinal de interrupção. Sem esses erros, main() retorna 0.

O retorno 0 indica término normal da execução, não que alguma porta esteja aberta. Erros individuais podem ter sido impressos e o programa ainda terminar com 0; essa é uma limitação para automação.

Linhas 44–47 — ponto de entrada. O guard __name__ == '__main__' chama main() quando executamos python main.py, mas não ao importar o módulo. SystemExit encaminha um código de erro ao processo chamador quando necessário.

11. connection.py: menu e TCP

Linhas 7–14 — menu. O dicionário traduz escolhas numéricas para nomes de protocolo. strip() aceita espaços em volta da escolha. O while True repete até obter uma opção válida; return encerra a função. Não há recursão nem acúmulo de chamadas a cada erro de digitação.

Criando e fechando o socket TCP

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
    sock.settimeout(timeout)
    result = sock.connect_ex((target_ip, port))
```

AF_INET seleciona IPv4. SOCK_STREAM, nesse contexto, seleciona TCP. O par (target_ip, port) é uma tupla com o destino. O with fecha o socket ao sair do bloco, inclusive se uma exceção interromper a operação.

O resultado é analisado depois do bloco. Portanto, quando a função classifica a tentativa, o socket já foi fechado. Não é uma conexão mantida para explorar ou conversar com o serviço.

Interpretando connect_ex()

A diferença prática em relação a connect() é que connect_ex() retorna códigos para vários erros de conexão, em vez de sempre levantá-los como exceções. Isso não garante que nenhuma exceção aconteça: resolução, argumentos e outros problemas ainda podem falhar.

0: sucesso. **ECONNREFUSED:** recusa. **ETIMEDOUT, EAGAIN e EWOULDBLOCK:** tratados pelo projeto como filtered/unknown. Em alguns sistemas EAGAIN e EWOULDBLOCK têm o mesmo valor; a duplicidade na tupla é inofensiva.

O módulo errno evita fixar números como 111, que não são portáveis entre sistemas. errno.errorcode.get(result, result) tenta traduzir o número restante para um nome simbólico; se não existir mapeamento, mantém o número.

O que esta função não faz

Não lê banner, não envia HTTP, não faz TLS, não mede latência explicitamente e não verifica se o serviço continua vivo depois do fechamento. Seu resultado se refere àquela tentativa e àquele instante.

Exercício mental: se o servidor é encerrado logo depois de connect_ex() retornar 0, o scan estava necessariamente errado? Não. O estado de rede pode mudar após uma observação correta.

12. connection.py: UDP e exceções

A função UDP cria SOCK_DGRAM e configura o prazo por socket. Depois associa o destino, envia b'probe' e tenta receber uma mensagem. O caminho de sucesso retorna open.

```
try:
    sock.connect((target_ip, port))
    sock.send(b'probe')
    sock.recv(4096)
    return 'open'
except ConnectionRefusedError:
    return 'closed'
except socket.timeout:
    return 'open|filtered'
```

A ausência de variável para o retorno de recv() é intencional: a implementação só verifica se houve resposta. Isso é suficiente para uma sondagem genérica, mas insuficiente para identificar ou validar um protocolo de aplicação.

Fluxo das exceções

Uma recusa é tratada na própria função. Um timeout também. Outro OSError sai da função, mas o with ainda fecha o socket. No main.py, o except interno imprime o erro para a porta e o protocolo e continua o laço.

Uma interrupção pelo usuário não é engolida por except OSError. KeyboardInterrupt herda de BaseException, não de OSError. Ela alcança o tratador externo e encerra o programa.

Melhorar a sonda sem mentir no resultado

Uma melhoria futura seria associar portas e protocolos conhecidos a payloads válidos, validar a resposta e manter uma opção de sonda genérica. O fato de a porta ser 53 pode orientar a tentativa DNS, mas não comprova a identidade do serviço.

Retransmitir com limite pode recuperar perdas ocasionais. Porém, repetições aumentam tráfego e não resolvem um servidor que ignora o payload. Nunca converta ausência de resposta em closed só para parecer mais preciso.

Cuidados de portabilidade: a implementação foi verificada em Linux neste ambiente. A forma e o momento em que erros ICMP chegam ao socket UDP podem variar em outros sistemas. Um teste local passando não valida automaticamente todos os firewalls, roteadores e sistemas operacionais.

13. Por que os testes são importantes

Para reproduzir os testes da versão estudada, execute a partir da raiz do projeto:

```
cd docs/snapshot
python -m unittest -v
```

Volte à raiz antes dos laboratórios: cd ../.. . A suíte da raiz pode divergir enquanto você modifica o programa; os cinco testes citados aqui se referem ao snapshot.

A verificação realizada para esta apostila executou cinco métodos de teste e terminou com OK. Eles cobrem cenários específicos; não são uma certificação de todas as redes possíveis.

TCP aberto e fechado: o teste faz bind em loopback com porta 0, lê a porta escolhida pelo sistema e chama listen(). Enquanto o socket servidor está aberto, espera open. Após fechá-lo, espera closed. Existe uma pequena janela em que outro processo poderia reutilizar a porta; isso seria uma fonte de instabilidade do teste.

UDP com resposta: uma thread recebe um datagrama e devolve o mesmo conteúdo ao remetente. O scanner deve retornar open. A thread é necessária porque o cliente precisa estar esperando enquanto o servidor recebe e responde.

UDP silencioso: um socket UDP permanece associado à porta, mas não responde. O scanner deve retornar open|filtered, provando que uma porta realmente ocupada pode produzir silêncio.

Menu: patch substitui input() por uma sequência de entradas. O teste fornece uma opção inválida e depois ' 3 ', e confere BOTH e a presença da terceira opção no texto. redirect_stdout evita poluir a saída dos testes.

Despacho do main: runpy executa main.py como ponto de entrada. As funções de sondagem são substituídas para contar chamadas sem depender da rede. São esperadas sete chamadas para cada protocolo selecionado. Isso testa a organização do programa, não o envio de pacotes.

O que ainda falta testar

DNS inválido, Ctrl+C, EOF, erro de rota, timeout TCP, erro UDP inesperado e entrada futura de portas ainda não têm cobertura específica nesta suíte. O teste TCP atual testa dois estados no mesmo método; separar casos pode melhorar o diagnóstico. O servidor UDP de teste usa thread: exceções nela deveriam ser propagadas explicitamente para o teste em uma evolução da suíte.

Distinção essencial: um mock confirma como você organizou chamadas; um servidor real local confirma comportamento de sockets naquele ambiente. São evidências complementares, não intercambiáveis.

14. Laboratório TCP: enxergue o serviço abrir e fechar

Os arquivos de laboratório ficam em docs/labs/. Todos usam 127.0.0.1. Não é necessário usar root nem expor o serviço à rede. Se uma porta já estiver ocupada, escolha outra porta não privilegiada e use o mesmo número no cliente.

Experimento A — conexão estabelecida

No terminal A, a partir da raiz do projeto:

```
python docs/labs/tcp_server.py
```

O servidor faz bind em 127.0.0.1:8000, chama listen() e aceita conexões. No terminal B:

```
python docs/labs/probe_local.py tcp 8000
```

Resultado esperado: open. O servidor pode registrar uma conexão e receber zero bytes: o scanner encerra sem enviar dados de aplicação. Isso não é um erro de HTTP; este laboratório nem implementa HTTP.

Experimento B — recusa

Encerre o terminal A com Ctrl+C. Repita a sonda no B. **Resultado esperado em loopback sem outro processo na porta:** closed. Se obtiver open, verifique se outro serviço ficou escutando.

Experimento C — use o main.py

O main.py completo em docs/snapshot/ não pergunta portas; usa a lista PORTS. Para incluir o laboratório nesse scan, edite a constante dessa cópia temporariamente para [8000], execute python docs/snapshot/main.py, selecione TCP e informe 127.0.0.1. Restaure a lista depois. O main.py da raiz pode continuar diferente enquanto você estuda. Não confunda mudar a configuração com mudar a função de sondagem.

Observe e explique

```
ss -lnt
```

Compare a listagem antes de iniciar, durante a escuta e depois do encerramento. O estado LISTEN representa o servidor aguardando conexões; não uma conexão já estabelecida com seu scanner.

Anote três evidências: resultado do scanner, porta em escuta no sistema e momento do encerramento. Aprender a cruzar evidências é mais importante que decorar '0 significa open'.

15. Laboratório UDP: uma porta aberta pode ficar calada

Experimento A — servidor de eco

No terminal A:

```
python docs/labs/udp_server.py
```

O servidor associa 127.0.0.1:9999, recebe datagramas e devolve o conteúdo para o endereço retornado por recvfrom(). No terminal B:

```
python docs/labs/probe_local.py udp 9999
```

Resultado esperado: open. Há evidência porque o socket do cliente recebeu um datagrama, não porque connect() teve sucesso.

Experimento B — escuta silenciosa

Encerre o servidor e reinicie em modo silencioso:

```
python docs/labs/udp_server.py --silent
python docs/labs/probe_local.py udp 9999
```

A segunda linha deve ser executada em outro terminal. **Resultado esperado:** open|filtered. O processo continua associado à porta e até recebe a mensagem, mas não responde. Você acabou de produzir o contraexemplo de 'se não respondeu, está fechado'.

Experimento C — sem servidor

Encerre o servidor silencioso e repita a sonda. Em Linux local, uma recusa normalmente resulta em closed. Em outros sistemas ou caminhos de rede, a informação pode não chegar da mesma forma e ocorrer timeout. Não trate esse comportamento como garantia universal.

Confirme o que mudou

```
ss -lnu
```

No modo de eco e no modo silencioso, a porta continua associada. UDP não usa listen()/accept() como TCP. O vínculo local permite receber datagramas; responder é uma decisão do programa.

Registre uma tabela sua: modo eco, silencioso e encerrado; resultado observado; existência da porta em ss; explicação. Não altere regras de firewall só para forçar estados: este laboratório demonstra a ambiguidade sem mudanças administrativas.

16. Observação de tráfego e roteiro de diagnóstico

Uma captura permite confrontar sua interpretação com o que efetivamente foi transmitido. Se Wireshark estiver disponível e você já tiver permissão de captura, use a interface de loopback e filtros de exibição como:

```
tcp.port == 8000
udp.port == 9999 || icmp
```

Esses são filtros de exibição do Wireshark, não comandos de shell nem filtros de captura tcpdump. A apostila não altera permissões de captura. Evite capturar interfaces com tráfego privado de outras pessoas.

O que procurar

No TCP aberto, procure SYN, SYN+ACK e ACK. Depois veja o encerramento; sua forma exata pode variar com o estado e com os dados pendentes. No fechado, procure a recusa. No UDP de eco, procure ida e volta. No silencioso, pode haver só a ida. Sem serviço, pode surgir ICMP port unreachable.

Os diagramas deste documento são conceituais. Nenhuma imagem de pacote aqui deve ser interpretada como captura efetuada para o projeto.

Diagnóstico em ordem

1. **Entrada:** qual endereço o programa mostrou? Está sondando o IP esperado? 2. **Resolução:** o nome resolveu? O resultado é IPv4? Há vários endereços possíveis? 3. **Serviço:** existe socket no protocolo certo e no endereço de escuta certo? 4. **Caminho:** há rota? Você está testando loopback, LAN ou uma tradução NAT? 5. **Política:** há descarte ou rejeição? Existe restrição por origem? 6. **Aplicação UDP:** o payload é reconhecido? O serviço deveria responder? 7. **Observação:** o timeout é adequado? O resultado é estável ao repetir com cuidado?

Se o programa imprimir error: ..., preserve a mensagem. Ela informa uma falha operacional diferente de closed. Corrigir a interpretação removendo erros da saída só esconde o problema.

Exercício: um servidor escuta apenas em 127.0.0.1. Por que outro computador na LAN não consegue acessá-lo pelo IP de rede do servidor? A resposta está no vínculo do socket ao endereço local, não necessariamente no firewall.

17. Limitações reais e caminho de evolução

A implementação atual é um bom ponto de partida educacional, não um substituto completo para ferramentas de inventário e diagnóstico consolidadas.

Endereçamento: apenas IPv4 e um endereço resolvido por alvo. Evolução: getaddrinfo(), escolha de família e deduplicação de endereços. IPv6 altera o formato do endereço do socket e usa ICMPv6, não simplesmente as mesmas regras IPv4.

Configuração: a lista de portas e o timeout estão no código. Evolução: argparse com --protocol tcp|udp|both, --ports e --timeout. Valide portas de 1 a 65535, remova duplicadas, valide timeout positivo e rejeite entradas com mensagem clara. Isso é proposta, não uma CLI já implementada.

Resultados: strings misturam estado e detalhes do erro. Evolução: um resultado estruturado com alvo, endereço, protocolo, porta, estado, detalhe e duração. JSON facilita automação; uma tabela facilita leitura humana.

Desempenho: tudo é sequencial. Evolução: ThreadPoolExecutor ou asyncio com concorrência limitada. Não crie uma thread por porta sem limite: você pode esgotar descritores, gerar perdas e distorcer resultados. Acrescente cancelamento e controle de taxa antes de aumentar a escala.

UDP: payload genérico, sem validação de resposta nem análise própria de ICMP. Evolução: sondas específicas com identificadores e respostas validadas; repetição limitada. Estados inconclusivos continuam necessários.

Precisão operacional: sem prazo total, sem controle explícito do tempo de DNS, sem retries e sem resumo final de falhas. O status de saída 0 não sinaliza erros individuais. Evolução: definir uma política documentada de conclusão parcial.

Ordem sugerida para aprender

1. Adicionar testes de falha de DNS e interrupção. 2. Separar configuração de lógica com argparse. 3. Criar um tipo de resultado e exportar JSON. 4. Medir duração real com relógio monotônico. 5. Adicionar concorrência pequena e comparar contra o sequencial. 6. Criar uma sonda UDP válida de um protocolo de laboratório.

Avance quando conseguir explicar e testar a etapa anterior. Mais velocidade não compensa uma classificação enganosa.

18. Exercícios de entendimento

Responda sem consultar o gabarito. Sempre separe observação e hipótese.

1. Você recebeu open para 443/tcp. Quais conclusões pode fazer sobre HTTPS, criptografia e vulnerabilidade?

2. O alvo responde TCP em 53. Isso confirma que 53/udp está aberto? Por quê?

3. O servidor UDP está rodando, mas o scanner mostra open|filtered. Liste pelo menos três explicações possíveis.

4. Em que momento connect() comprova estabelecimento remoto no TCP? Por que o mesmo raciocínio não serve para UDP?

5. Qual o problema de chamar socket.setdefaulttimeout(0.1) depois de criar o primeiro socket?

6. Em probes.append(('tcp', connection.connection_tcp)), por que não há parênteses depois de connection_tcp?

7. Quais são a ordem e a quantidade de sondagens em BOTH com a lista atual? O que acontece se uma delas levanta OSError?

8. A resolução do alvo falha. Isso deve ser exibido como todas as portas fechadas? O TIMEOUT atual controla essa resolução?

9. recv(4096) recebeu um datagrama UDP vazio. O código retorna open ou open|filtered? E se o datagrama for maior que o buffer?

10. Por que o teste usa bind(('127.0.0.1', 0))? Por que consegue testar TCP aberto sem chamar accept()?

11. Quais testes usam rede real local e quais substituem funções? O que isso muda sobre a evidência produzida?

12. O programa terminou com código 0. Isso comprova que todas as sondagens foram bem-sucedidas e que existe alguma porta aberta?

Desafios de implementação

A. Validação: crie uma função que receba uma lista textual de portas e devolva inteiros únicos, preservando a ordem. Teste vazios, duplicados, zero, negativos, 65536 e texto inválido antes de integrar.

B. Tempo: registre duração por sonda usando time.perf_counter(). Explique por que não usar o horário de parede para medir intervalos.

C. Resultado: troque o retorno textual por uma dataclass sem alterar a semântica dos estados. Atualize os testes antes de mudar a apresentação.

19. Gabarito comentado

1. Apenas que a conexão TCP foi estabelecida naquele instante e caminho. O número 443 não prova HTTPS, TLS ou vulnerabilidade. Seriam necessárias verificações de aplicação independentes.

2. Não. TCP e UDP são protocolos diferentes, com sockets e políticas de firewall distintos. O número da porta não os funde.

3. O serviço pode ignorar b'probe', a resposta pode ter sido perdida ou bloqueada, o timeout pode ser curto, ou o retorno pode usar outro endpoint não aceito pelo socket conectado. Estar em execução também não garante bind no endereço esperado.

4. No caminho normal de sucesso de connect()/connect_ex() TCP, a pilha estabeleceu a conexão. No UDP, connect() só associa o par remoto localmente, sem handshake.

5. O socket já criado mantém sua configuração anterior. A chamada altera o padrão para futuros sockets e pode afetar outras partes do programa. settimeout() atua diretamente no socket pretendido.

6. A lista armazena a função para executar depois. Com parênteses, você a chamaria imediatamente e tentaria armazenar seu resultado; sem argumentos, essa chamada também falharia para a assinatura atual.

7. São quatorze sondagens: TCP e UDP de cada uma das sete portas, na ordem de PORTS. OSError da tentativa vira error: ... e o laço prossegue. Ctrl+C é tratado externamente e encerra.

8. Não houve endereço resolvido para iniciar o scan; é um erro de resolução. TIMEOUT é passado aos sockets de sondagem, não a gethostbyname().

9. open, pois não há verificação de conteúdo nem de verdade lógica do retorno. Um datagrama maior que o limite pode ser truncado; isso inviabilizaria validar uma mensagem completa sem tratamento adicional.

10. Porta 0 pede uma porta livre ao sistema. listen() permite que o kernel estabeleça conexões e as deixe na fila antes de accept(); o scanner só precisa do estabelecimento.

11. TCP, eco UDP e UDP silencioso usam sockets reais em loopback. Menu usa input substituído; despacho usa funções de sonda substituídas. Estes verificam decisões do programa, não comunicação real.

12. Não. O código 0 indica término normal de main(). As portas podem estar todas fechadas e erros individuais podem ter sido registrados sem alterar o código final.

Critério de domínio

Você entendeu o núcleo quando consegue prever os três modos do laboratório UDP, explicar o handshake TCP sem atribuí-lo ao Python e justificar por que erro, fechado e inconclusivo não são sinônimos.

20. Referências e revisão rápida

Fontes primárias para continuar estudando

Python — socket, versão 3.14: documentação de socket(), gethostbyname(), connect(), connect_ex(), settimeout() e setdefaulttimeout(). Observe as notas de comportamento dependente da plataforma. https://docs.python.org/3.14/library/socket.html

Python — errno: nomes simbólicos de erros do sistema operacional. https://docs.python.org/3.14/library/errno.html

Python — unittest e unittest.mock: execução de testes, assertions e substituição controlada de dependências. https://docs.python.org/3.14/library/unittest.html https://docs.python.org/3.14/library/unittest.mock.html

RFC 9293 — TCP: especificação consolidada do transporte TCP, incluindo estados, estabelecimento e tratamento de reset. https://www.rfc-editor.org/rfc/rfc9293.html

RFC 768 — UDP: formato do datagrama e propriedades mínimas do protocolo. https://www.rfc-editor.org/rfc/rfc768.html

RFC 792 — ICMP para IPv4: mensagens de controle e Destination Unreachable. Documenta a base histórica; há atualizações posteriores. https://www.rfc-editor.org/rfc/rfc792.html

Glossário essencial

Endpoint: endereço e porta de um participante. **Bind:** associação local do socket. **Listener:** socket TCP aguardando conexões. **Handshake:** estabelecimento com troca de mensagens. **Payload:** dados carregados após os cabeçalhos. **Timeout:** prazo local de espera. **RST:** reset TCP. **ICMP:** mensagens de controle/erro da camada IP. **Loopback:** comunicação no próprio host. **Sonda:** estímulo usado para observar uma resposta.

Folha de bolso

TCP conectado: evidência de abertura. UDP connect(): nenhuma prova remota. UDP com resposta: evidência de abertura. Silêncio: não confundir com fechado. Número de porta: não prova serviço. Serviço acessível: não prova vulnerabilidade. Teste local: não prova portabilidade universal.

O apêndice a seguir contém o código completo numerado, extraído dos arquivos do projeto durante a geração. É um snapshot para estudo: alterações futuras no projeto não atualizam este PDF automaticamente.

Apêndice A — main.py completo

Arquivo: docs/snapshot/main.py. Numeração original preservada. Linhas longas podem continuar visualmente com o marcador |.

```
 1  import socket
 2
 3  import connection
 4
 5
 6  PORTS = [53, 22, 80, 111, 139, 443, 32768]
 7  TIMEOUT = 1.0
 8
 9
10  def main():
11      try:
12          print('Scan only hosts you own or have permission to test.')
13          connection_type = connection.connection_choice()
14          target = input('Insert the target IP or hostname: ').strip()
15          target_ip = socket.gethostbyname(target)
16
17          print('-' * 50)
18          print(f'Scanning {target_ip} ({connection_type})...')
19          print('-' * 50)
20
21          probes = []
22          if connection_type in ('TCP', 'BOTH'):
23              probes.append(('tcp', connection.connection_tcp))
24          if connection_type in ('UDP', 'BOTH'):
25              probes.append(('udp', connection.connection_udp))
26
27          for port in PORTS:
28              for protocol, probe in probes:
29                  try:
30                      state = probe(target_ip, port, TIMEOUT)
31                  except OSError as error:
32                      state = f'error: {error}'
33                  print(f'{port}/{protocol}: {state}')
34
35      except socket.gaierror as error:
36          print(f'Cannot resolve target: {error}')
37          return 1
38      except (KeyboardInterrupt, EOFError):
39          print('\nProgram closed by the user!')
40          return 130
41      return 0
42
43
44  if __name__ == '__main__':
45      exit_code = main()
46      if exit_code:
47          raise SystemExit(exit_code)
```

Apêndice B — connection.py completo

Arquivo: docs/snapshot/connection.py. Numeração original preservada. Linhas longas podem continuar visualmente com o marcador |.

```
"""IPv4 probes. Use only against targets you are authorized to test."""

import errno
import socket


def connection_choice():
    choices = {'1': 'TCP', '2': 'UDP', '3': 'BOTH'}
    while True:
        print('Choose connection type:\n1. TCP\n2. UDP\n3. BOTH')
        choice = input('Enter your choice (1, 2 or 3): ').strip()
        if choice in choices:
            return choices[choice]
        print('Invalid choice. Please select 1, 2 or 3.')


def connection_tcp(target_ip, port, timeout=1.0):
    """A successful TCP handshake means open; timeout is inconclusive."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.settimeout(timeout)
        result = sock.connect_ex((target_ip, port))

    if result == 0:
        return 'open'
    if result == errno.ECONNREFUSED:
        return 'closed'
    if result in (errno.ETIMEDOUT, errno.EAGAIN, errno.EWOULDBLOCK):
        return 'filtered/unknown'
    return f'error: {errno.errorcode.get(result, result)}'


def connection_udp(target_ip, port, timeout=1.0):
    """Generic UDP probe: silence cannot distinguish open from filtered.

    Services may ignore this payload. ICMP error delivery is OS-dependent.
    """
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
        sock.settimeout(timeout)
        try:
            sock.connect((target_ip, port))
            sock.send(b'probe')
            sock.recv(4096)
            return 'open'
        except ConnectionRefusedError:
            return 'closed'
        except socket.timeout:
            return 'open|filtered'
```

Apêndice C — testes completos

Arquivo: docs/snapshot/test_connection.py. Numeração original preservada. Linhas longas podem continuar visualmente com o marcador |.

```
 1  import contextlib
 2  import io
 3  import socket
 4  import threading
 5  import unittest
 6  from unittest.mock import patch
 7
 8  import connection
 9
10
11  class ScannerTests(unittest.TestCase):
12      def test_tcp_open_and_closed(self):
13          with socket.socket() as server:
14              server.bind(('127.0.0.1', 0))
15              port = server.getsockname()[1]
16              server.listen()
17              self.assertEqual(connection.connection_tcp('127.0.0.1', port, 0.2), 'open')
18          self.assertEqual(connection.connection_tcp('127.0.0.1', port, 0.2), 'closed')
19
20      def test_udp_response_is_open(self):
21          with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as server:
22              server.bind(('127.0.0.1', 0))
23              server.settimeout(2)
24              def reply():
25                  data, peer = server.recvfrom(4096)
26                  server.sendto(data, peer)
27              worker = threading.Thread(target=reply)
28              worker.start()
29              try:
30                  self.assertEqual(connection.connection_udp('127.0.0.1', server.getsockname()[1],
    | 0.5), 'open')
31              finally:
32                  worker.join(timeout=3)
33
34      def test_udp_silence_is_inconclusive(self):
35          with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as server:
36              server.bind(('127.0.0.1', 0))
37              self.assertEqual(connection.connection_udp('127.0.0.1', server.getsockname()[1],
    | 0.05), 'open|filtered')
38
39      def test_main_dispatches_selected_protocols(self):
40          import runpy
41          for choice, tcp_calls, udp_calls in [('1', 7, 0), ('2', 0, 7), ('3', 7, 7)]:
42              with self.subTest(choice=choice), \
43                      patch('builtins.input', side_effect=[choice, '127.0.0.1']), \
44                      patch.object(connection, 'connection_tcp', return_value='open') as tcp, \
45                      patch.object(connection, 'connection_udp', return_value='open|filtered') as
    | udp, \
46                      contextlib.redirect_stdout(io.StringIO()):
47                  runpy.run_path('main.py', run_name='__main__')
48                  self.assertEqual(tcp.call_count, tcp_calls)
49                  self.assertEqual(udp.call_count, udp_calls)
50
51      def test_menu_retries_and_offers_both(self):
52          output = io.StringIO()
53          with patch('builtins.input', side_effect=['invalid', ' 3 ']),
    | contextlib.redirect_stdout(output):
54              self.assertEqual(connection.connection_choice(), 'BOTH')
55          self.assertIn('3. BOTH', output.getvalue())
56
57
58  if __name__ == '__main__':
59      unittest.main()
```

Apêndice D — servidor TCP local

Arquivo: docs/labs/tcp_server.py. Numeração original preservada. Linhas longas podem continuar visualmente com o marcador |.

```
 1  """Local TCP listener for the study guide; no application protocol."""
 2  import socket
 3
 4
 5  def main():
 6      with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
 7          server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
 8          server.bind(('127.0.0.1', 8000))
 9          server.listen()
10          print('TCP ready on 127.0.0.1:8000', flush=True)
11          while True:
12              client, peer = server.accept()
13              with client:
14                  client.settimeout(2)
15                  try:
16                      data = client.recv(4096)
17                      print(f'{peer}: {len(data)} application bytes', flush=True)
18                  except socket.timeout:
19                      print(f'{peer}: no data before timeout', flush=True)
20
21
22  if __name__ == '__main__':
23      try:
24          main()
25      except KeyboardInterrupt:
26          print('\nServer stopped.')
```

Leitura guiada: bind fixa o endpoint local; listen prepara a fila TCP; accept devolve um novo socket para o cliente. O socket de escuta continua aberto. recv vazio indica que o outro lado encerrou seu envio sem dados pendentes.

Apêndice E — servidor UDP local

Arquivo: docs/labs/udp_server.py. Numeração original preservada. Linhas longas podem continuar visualmente com o marcador |.

```
 1  """Local UDP echo server; --silent receives but does not reply."""
 2  import argparse
 3  import socket
 4
 5
 6  def main():
 7      parser = argparse.ArgumentParser(description=__doc__)
 8      parser.add_argument('--silent', action='store_true')
 9      args = parser.parse_args()
10      with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as server:
11          server.bind(('127.0.0.1', 9999))
12          print(f'UDP ready on 127.0.0.1:9999; silent={args.silent}', flush=True)
13          while True:
14              data, peer = server.recvfrom(4096)
15              print(f'{peer}: {len(data)} bytes received', flush=True)
16              if not args.silent:
17                  server.sendto(data, peer)
18
19
20  if __name__ == '__main__':
21      try:
22          main()
23      except KeyboardInterrupt:
24          print('\nServer stopped.')
```

Leitura guiada: não há listen nem accept. recvfrom devolve bytes e o endereço do remetente; sendto usa esse endereço para a resposta. --silent mantém a recepção, mas suprime o envio.

Apêndice F — cliente do laboratório

Arquivo: docs/labs/probe_local.py. Numeração original preservada. Linhas longas podem continuar visualmente com o marcador |.

```
"""Probe one loopback port using the project's actual scan functions."""
import argparse
from pathlib import Path
import sys

# The script lives two directories below the project modules.
sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
import connection


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('protocol', choices=['tcp', 'udp'])
    parser.add_argument('port', type=int)
    args = parser.parse_args()
    if not 1 <= args.port <= 65535:
        parser.error('port must be between 1 and 65535')
    probe = connection.connection_tcp if args.protocol == 'tcp' else connection.connection_udp
    print(probe('127.0.0.1', args.port, 1.0))


if __name__ == '__main__':
    main()
```

Leitura guiada: argparse valida protocolo e porta; o alvo é fixo em loopback. O ajuste de sys.path permite importar connection.py da raiz quando o script é executado por caminho. Isso é uma conveniência deste laboratório, não uma exigência dos sockets.

Apêndice G — evidências de execução

Saída real da execução de docs/verify_study_guide.py neste ambiente Linux. Os laboratórios usam o connection.py da raiz e os testes abaixo usam a cópia em docs/snapshot/. Servidores temporários são encerrados ao final.

```
TCP ready on 127.0.0.1:8000
tcp/8000: open

tcp/8000 after shutdown: closed

UDP ready on 127.0.0.1:9999; silent=False
udp/9999: open

UDP ready on 127.0.0.1:9999; silent=True
udp/9999: open|filtered

udp/9999 after shutdown: closed

test_main_dispatches_selected_protocols
     | (test_connection.ScannerTests.test_main_dispatches_selected_protocols) ... ok
test_menu_retries_and_offers_both (test_connection.ScannerTests.test_menu_retries_and_offers_both) ...
     | ok
test_tcp_open_and_closed (test_connection.ScannerTests.test_tcp_open_and_closed) ... ok
test_udp_response_is_open (test_connection.ScannerTests.test_udp_response_is_open) ... ok
test_udp_silence_is_inconclusive (test_connection.ScannerTests.test_udp_silence_is_inconclusive) ... ok

----------------------------------------------------------------------
Ran 5 tests in 0.057s

OK
```

Os estados após encerramento refletem este ambiente local, não uma garantia para todos os sistemas. O main.py atual da raiz foi preservado e não é o ponto de entrada completo avaliado aqui.

Identificação do snapshot

snapshot/main.py — 47 linhas — SHA-256: 2a2c1210ef049d2484f3695d96d9de46ecb71caed7d9fe3f08c3c76cde0d2062

snapshot/connection.py — 47 linhas — SHA-256: d9cfdf4fcf692bdd29bea32aa987b52ed966a0b50138af7eddf24058ece7eaea

snapshot/test_connection.py — 59 linhas — SHA-256: 8940c5e620e0cbf089c18a66c74ef746f1a9f37f61063c78a2ef5723a4ffc1fb

labs/tcp_server.py — 26 linhas — SHA-256: 131671a49aeb93b683d91b811dc1a74fd33d79af4472f92b7871cfae38c4c926

labs/udp_server.py — 24 linhas — SHA-256: a2a6d0d0347283f65b9c79fafc2fb16e8ffee8a95ac99b0f0ad14d2543b590af

labs/probe_local.py — 23 linhas — SHA-256: 628090233f542888c28a6bb5a2343668ae01d5fefc245db6c72ca0cb7ce53043